Identifying LLMs from Generated Responses

CMPSC 448 Midterm Project

Allegra Francis [Team leader] **(Recitation Section: 004R)** and Amal Palackal (**Recitation Section: 003R)**

October 11, 2026

Github repository: https://github.com/amalsony/llm-fingerprinting

1. Problem Definition and Dataset

Our project tests whether a convolutional neural network (CNN) and a recurrent neural network implemented as an LSTM can identify the language model that generated a response. We used a subset of the RAID dataset (Dugan, et al.) with 1,600 responses from four generators across four writing domains. With all domains represented during training, test accuracy is 70.625% for the CNN and 57.500% for the LSTM. When the test domain is excluded from training and validation, accuracy on the same 160 test responses falls to 30.625% and 23.750%. These results indicate limited transfer across domains in this setup, although the domain exclusion experiment also uses less training data.

Our classes were ChatGPT, Cohere Chat, Llama Chat, and Mistral Chat. RQ1 asks whether output text identifies its generator. RQ3 asks whether that identification transfers to a domain absent during training. RQ2 examines prompt information, and RQ4 analyzes distinguishing text characteristics.

We used publicly released RAID data, specifically the labeled training partition without adversarial attacks. We did not generate new responses or call LLM APIs to collect data. The official test partition is not used because its labels are hidden. The selected RAID identifiers are chatgpt, cohere-chat, llama-chat, and mistral-chat, representing four different model families. These are the historical RAID generators, not current versions of those families.

Table 1. Final dataset composition. Every source has one response from each of the four generators.

Domain	Matched sources	Responses	Responses per generator
News	100	400	100
Poetry	100	400	100
Recipes	100	400	100
Reviews	100	400	100
Total	400	1600	400

We retain attack=none, decoding=sampling, and repetition_penalty=no. A source is eligible only if all four model responses are nonempty and their prompts match exactly. Duplicate sources,

normalized prompts, or normalized outputs across sources are removed by retaining a complete group in deterministic hash order. Normalization collapses whitespace and folds case solely for duplicate checks; the saved text keeps its original formatting.

The filter yields 6,266 complete sources before deduplication. After deduplication, eligible counts are 1,737 news, 1,703 poetry, 1,771 recipes, and 941 reviews. We select 100 per domain using seed 42. A source-file checksum, selected IDs, filters, and split assignments are recorded in the dataset manifest.

2. CNN and RNN (LTSM) Implementation and Design

Both classifiers use the response text alone for RQ1 and RQ3. We tokenize words, punctuation, and newlines, build the vocabulary using only training text, and keep the first 256 tokens of each response. Both models learn 128-dimensional embeddings from scratch. The CNN uses 128 filters at each width of 3, 4, and 5 tokens, followed by maximum pooling. The LSTM has one layer with 128 hidden units and averages its token states before classification.

We train with cross-entropy loss and AdamW, using a learning rate of 0.001, batch size of 32, and dropout of 0.3. Training runs for up to 15 epochs, stopping after four epochs without validation improvement. We evaluate the checkpoint with the best validation macro F1. All runs use seed 42. We report accuracy and macro F1, which weights all four generators equally; chance accuracy is 25%.

2.1 CNN

Both models learn randomly initialized 128-dimensional word embeddings jointly with the classifier. The CNN applies 128 one-dimensional filters at each width of 3, 4, and 5 tokens. ReLU activations and maximum pooling produce 384 features, followed by dropout and a four-class linear layer. Pooling excludes windows beyond the response; very short responses use one padded window. The CNN has 1,635,844 parameters.

2.2 RNN (LTSM)

The RNN is a single forward LSTM layer with 128 hidden units. We average hidden states over real tokens, apply dropout, and use a four-class linear layer. Padding states are excluded from the average. Because the LSTM is unidirectional, later padding cannot affect earlier real-token states. The LSTM has 1,569,924 parameters. Neither model uses pretrained embeddings or a pretrained LLM.

3. Research Question 1: *Can we identify which LLM generated a response?*

3.1 Results

Table 3. RQ1 output-only performance on 160 responses from 40 test sources.

Classifier	Accuracy	Macro F1	Selected epoch	Epochs run
CNN	70.625%	0.7012	6	10

LSTM	57.500%	0.5536	10	14

Table 4. RQ1 precision, recall, and F1 by generator. Each generator has 40 test responses.

Generator	CNN Precision	CNN Recall	CNN F1	LSTM Precision	LSTM Recall	LSTM F1
ChatGPT	0.698	0.925	0.796	0.567	0.850	0.680
Cohere	0.862	0.625	0.725	0.580	0.725	0.644
Llama	0.714	0.750	0.732	0.654	0.425	0.515
Mistral	0.583	0.525	0.553	0.500	0.300	0.375

3.2 Analysis

The CNN correctly classifies 113 of 160 test responses, compared with 92 for the LSTM. Both exceed the 25% chance-accuracy reference on this split. The CNN achieves higher accuracy and macro F1, but this single run does not establish that CNNs are generally superior to LSTMs.

ChatGPT has the highest recall in both classifiers: 92.5% for the CNN and 85.0% for the LSTM. The CNN identifies Cohere with high precision (86.2%) but lower recall (62.5%), so its Cohere predictions are relatively reliable while missing some true Cohere responses. Mistral is hardest for both models, with recalls of 52.5% and 30.0%.

The confusion matrices show the errors behind these averages. The CNN labels 11 of 40 Cohere responses as Mistral, and labels 10 Mistral responses as ChatGPT and 9 as Llama. The LSTM labels 13 of 40 Mistral responses as ChatGPT and 11 Llama responses as Cohere. These patterns show uneven class separation; they do not by themselves identify the linguistic features responsible.

Within this balanced four-generator dataset and fixed mixed-domain split, generated responses contain enough signal for both neural classifiers to perform above chance. The CNN is stronger in this run. The result is limited to the first 256 tokens of these responses: 875 of 1,280 training responses, 106 of 160 validation responses, and 115 of 160 test responses exceed that cap. It does not demonstrate reliable attribution across unseen domains, unknown generators, or newer model versions.

4. Research Question 2: *Does the user's prompt help identify the LLM?*

We compared CNN and LSTM classifiers using three input conditions: prompt only, response only, and prompt plus response. We used the same dataset and source-group splits described

above. Our main questions were whether adding the prompt improves classification overall and whether its benefit differs across domains.

We retained the first 128 prompt tokens and the first 256 response tokens. The combined condition included both portions with a newline separator, giving a maximum of 385 tokens. Response-only and combined inputs therefore contained exactly the same response text. This kept the comparison fair by preventing prompt tokens from displacing response tokens.

Each classifier was trained separately for each condition using the shared model implementations and training settings. We fitted each vocabulary only on the text visible to that condition during training. This differs slightly from RQ1, where the vocabulary was fitted on full training responses before truncation. We therefore trained a new response-only baseline rather than copying the RQ1 scores. These runs used CPU, so device differences may also contribute to differences from RQ1.

Classifier	Input condition	Accuracy	Macro F1
CNN	Prompt only	25%	.1983
LSTM	Prompt only	25%	.1922
CNN	Response only	73.125%	.7331
LSTM	Response only	53.750%	.5192
CNN	Both	70.000%	.7028
LSTM	Both	52.500%	.5158

Adding the prompt did not improve overall performance in this run. CNN accuracy decreased by 3.125 percentage points, while LSTM accuracy decreased by 1.250 points. Macro F1 also decreased for both classifiers.

We estimated uncertainty using 2,000 paired bootstrap resamples of complete source groups. The 95% interval for the accuracy change was −11.25 to +3.75 percentage points for the CNN and −8.75 to +6.25 for the LSTM. Both intervals include zero. The results therefore show no clear overall benefit from adding the prompt, but they do not establish that prompts consistently reduce performance.

Accuracy change from adding the prompt, calculated as prompt + response minus response only per domain

Domain	CNN change	CNN 95% int	LSTM change	LSTM 95%

News	-22,50	[-37.50, -7.50]	-7.50	[-20.00, +5.00]
Poetry	+7.50	[-5.00,+22.50]	+10.00	[0.00, +20.00]
Recipes	+7.50	[-2.50, +17.50]	-5.00	[-22.50, +12.50]
Reviews	-5.00	[-15.00, +7.50]	-2.50	[-17.50, +12.50]

Poetry improved for both classifiers, while recipes improved only for the CNN. Both classifiers performed worse on news and reviews after adding the prompt. The largest decrease was the CNN on news, where accuracy fell by 22.5 percentage points and the interval stayed below zero.

4.2 Interpretation These observations suggest that the effect of prompt context may depend on the task, but the domain comparisons are exploratory. Each domain contains only 10 independent test sources, with four responses per source. The intervals describe uncertainty over test sources for the fixed trained models; they do not include variation across training seeds. We also did not formally test differences between domain gains or adjust for multiple comparisons. This analysis evaluates performance within domains represented during training, unlike the domain exclusion experiment in RQ3.

Prompt-only accuracy was exactly 25% because each prompt appears with all four generator labels. A deterministic classifier makes the same prediction for those four identical inputs and gets one correct. This is a property of the balanced dataset rather than evidence that training failed. Macro F1 can differ from 0.25 because it depends on the distribution of predicted labels. The dataset therefore provides a controlled test of prompt context, but cannot reveal which model a user deliberately chose to ask.

5. Research Question 3: *Do LLM fingerprints generalize across tasks or domains?*

We hold out each domain in turn and train fresh CNN and LSTM models on the other three domains. Each run uses 960 training responses and 120 validation responses, then tests on all 400 responses in the held-out domain. We rebuild the vocabulary using only training text and retain the RQ1 model and training settings.

5.1 Results

Held-out domain	CNN accuracy	LSTM accuracy
News	32.75%	25.00%
Poetry	27.00%	28.75%

Recipes	25.00%	25.00%
Reviews	38.00%	30.50%
Mean	30.69%	27.31%

5.2 Analysis

Accuracy stays close to chance, with reviews showing the best transfer. Both classifiers label every unseen recipe response as Cohere, producing 25% accuracy but only 0.10 macro F1. This illustrates why accuracy alone can hide a failure to distinguish classes.

For a direct comparison with RQ1, we also evaluate the RQ3 predictions on the same 160 test responses used in RQ1. CNN accuracy falls from 70.63% to 30.63%, a drop of 40 percentage points. LSTM accuracy falls from 57.50% to 23.75%, a drop of 33.75 points. These matched results use a smaller subset than the table above.

The classifiers therefore transfer poorly to unseen domains in this setup. However, RQ3 also uses fewer training responses than RQ1, so the decline cannot be attributed entirely to domain differences. A comparison with equal training-set sizes would help separate these effects.

6. Research Question 4: *What characteristics distinguish different LLMS?*

We analyzed the full original training responses for differences in length, vocabulary, lexical diversity, and formatting. Features included word and character counts, average word length, approximate sentence length, punctuation, newlines, bullets, headings, code fences, and uppercase letters. We also examined frequent words and summarized features within each domain. This descriptive analysis used training responses only.

Table 7. Mean length and lexical diversity of training responses.

Generator	Mean word count	Type-token ration	First 100 words
ChatGPT	320.4	.550	.703
Cohere	186.9	.595	.666
Llama	270.0	.530	.677
Mistrals	211.7	.566	.674

ChatGPT produced the longest responses on average, followed by Llama, Mistral, and Cohere. Type-token ratio measures the number of distinct words divided by the total word count. The raw measure generally decreases with response length, so differences can partly reflect how much each model writes. Measuring the first 100 words keeps the word budget constant, although it excludes responses shorter than 100 words.

These results show that the generators differ in their responses, but descriptive differences alone do not establish which features the classifiers use. The feature analysis also covers full responses, while the classifiers see only the first 256 tokens.

We retrained both classifiers under three conditions: original responses, formatting removed, and token order shuffled. Each condition began with the same first 256 response tokens used for the RQ2 response-only baseline.

Formatting removal preserved the visible words and capitalization while removing punctuation and line breaks. Token shuffling preserved the same tokens but rearranged them in a deterministic random order. Neither transformation introduced additional response text from beyond the original input window. We retained the original splits and training settings, fitting a separate vocabulary for each transformed condition.

Table 8. RQ4 performance after transforming the responses.

Classifier	Condition	Accuracy	Macro F1
CNN	Original response	73.125%	.7331
LSTM	Original response	53.750%	.5192
CNN	Formatting removed	69.375	.6927
LSTM	Formatting removed	55.625	.5391
CNN	Token order shuffled	51.875%	.4972
LSTM	Token order shuffled	60.625	.5924

The original-response runs reproduced the RQ2 response-only baseline. Removing formatting reduced CNN accuracy by 3.75 percentage points. Shuffling token order caused a larger decrease of 21.25 points. This suggests that ordered patterns were useful to the CNN under this training setup, while formatting had a smaller effect.

The LSTM showed a different pattern. Accuracy increased by 1.875 percentage points after formatting removal and by 6.875 points after shuffling. Macro F1 also increased under both transformations. These results do not support token order as essential for this retrained LSTM. Differences in optimization and training variability could contribute to the improvements, so repeated runs would be needed before drawing a broader conclusion.

6.2 Interpretation The descriptive analysis identified linguistic and structural differences, while the transformation experiments tested whether changing some of those signals affected classification. The CNN's decrease after shuffling is consistent with its use of local ordered patterns. However, we cannot identify a single responsible feature from these experiments.

Shuffling changes syntax, local phrases, and layout at the same time. Removing formatting also changes token counts. The results therefore show sensitivity to these transformations rather than isolating individual causal signals. Since the classifiers were retrained on transformed text, the experiments assess what they can learn from the remaining information, rather than directly measuring the original classifiers' feature reliance. All results use one training seed, and we did not estimate statistical uncertainty for the ablation differences.

7. Limitations and Lessons Learned

The CNN performs better than the LSTM on the mixed-domain test set, but neither reliably identifies generators across unseen domains. The main lessons are to keep related responses together when splitting data, select models using validation performance, and examine class-level errors alongside overall accuracy.

This study uses one seed, a small balanced subset, four historical RAID generators, and four English writing domains. It does not cover code, mathematics, unknown generators, or current model versions. We report point estimates without confidence intervals or significance claims. Responses sharing a source are related, and RQ3 folds have overlapping training sets, so treating every prediction as an independent observation would overstate certainty.

Our results are limited to one seed, four RAID generators, and a small dataset. Most RQ1 test responses exceed the 256-token limit, so the models mainly see response beginnings. Larger datasets, repeated runs, and longer inputs would help assess how well the findings generalize.

RQ2 showed no clear overall improvement from adding prompts, although the direction of the change varied by domain. RQ4 showed that removing formatting and shuffling tokens affected the classifiers differently. Ordered patterns appeared useful for the CNN, while the retrained LSTM improved after both transformations in this run.

These experiments reinforced the importance of controlling how much response text each condition receives. They also showed that observable differences between generators do not automatically explain classifier predictions. Repeated training seeds and larger domain samples would help determine whether these patterns are stable

Contributions

Allegra Francis completed RQ2 and RQ4 and Amal Palackal completed RQ1 and RQ3. As permitted by the project guidelines, we used ChatGPT and Codex to help plan and implement this project.

References

Dugan et al. (2024). RAID: A Shared Benchmark for Robust Evaluation of Machine-Generated Text Detectors. ACL. https://aclanthology.org/2024.acl-long.674/

RAID dataset. https://huggingface.co/datasets/liamdugan/raid